Voice-Controlled Agentic Development: A Secure Control-Plane/Execution-Plane Architecture

Practical architecture for voice-driven orchestration, remote execution, and durable knowledge capture

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

Voice interaction changes the practical shape of agentic software development. A developer can remain in a conversational interface while coding agents inspect repositories, run commands, test changes, review design decisions, and produce durable artifacts on a remote development machine. The useful architecture is therefore not a single autonomous agent with unrestricted access. It is a layered system that separates conversational control from execution, gives each agent a defined role, and keeps privileged actions inside explicit technical boundaries.

This whitepaper proposes a concrete architecture built around a Windows laptop as the control plane and an Ubuntu development box as the execution plane. ChatGPT Voice in Codex or Work provides the spoken interface. Codex connects to the Ubuntu host through Remote SSH and serves as the primary implementation agent and task orchestrator. Claude Code runs on the Ubuntu host as a second coding and review agent. Git records changes and supports parallel worktrees; containers isolate autonomous work without requiring a full virtual machine for every task. Skills provide reusable workflows for tasks such as turning a voice conversation into a whitepaper. The host operating system retains ordinary privilege boundaries, and the design avoids passwordless root access for agents.

The contribution is a practical control-plane/execution-plane pattern for voice-driven agentic development that preserves speed while making execution reversible, inspectable, and bounded.

1. Motivation

The initial problem is interaction friction. Voice conversation is effective for developing ideas because it supports interruption, correction, and rapid exploration. It is less effective when the useful result remains trapped inside a conversation. A developer may discuss an architecture, discover a design constraint, or refine a research idea while away from a desk; converting that material into code or a durable document often requires a second session and manual transfer of context.

Agentic coding tools create an opportunity to remove that break. OpenAI currently documents Voice in Work and Codex for the ChatGPT desktop app on Windows and macOS. Voice can start tasks, check progress, and coordinate work using the tools and permissions available to the selected experience [1]. OpenAI also documents Remote SSH for Codex, including the ability for the desktop app to detect SSH hosts and run Codex threads on remote development machines [2]. Those two capabilities support a useful separation: the laptop handles conversation and human judgment, while the remote Linux machine handles computation and tool execution.

The same architecture also addresses a second concern: autonomy. Coding agents are increasingly useful when allowed to run tests, install project dependencies, edit many files, or delegate subtasks. Removing every permission prompt can improve throughput, but it also removes a safety boundary. The system therefore needs a place where high-autonomy execution is acceptable without granting the agent unrestricted control over the host operating system or unrelated data.

A dedicated Ubuntu development box provides that place. It can expose repositories and development tools to agents while keeping the developer's primary laptop separate. Containers can provide additional isolation for tasks that need broad command execution. Git records what changed. The host can remain unprivileged even when individual containers are disposable and highly permissive.

2. Design Goals

The architecture follows six design goals derived from the conversation and current product capabilities.

2.1 Voice should operate as a control surface

The developer should be able to state intent in natural language, ask follow-up questions, redirect work, and request artifacts without manually moving between terminals for routine steps. Voice should control tasks rather than emulate keystrokes.

2.2 Execution should stay where the development environment already exists

Source code, build tools, credentials, local services, and compute should remain on the Ubuntu development box. The Windows laptop should not need to mirror the complete environment or copy source code back and forth for each task.

2.3 Git should preserve history, not serve as the transport

Git is the durable audit and recovery layer. The remote connection should operate directly on the development box. Agents edit files and run tools on that host, then commit or push when a useful checkpoint exists. This avoids a fragile workflow in which each conversational action depends on a push-pull cycle between two machines.

2.4 High autonomy should exist inside a boundary

The host should not give the agent blanket passwordless sudo access. Broad autonomy belongs inside a sandbox, container, disposable environment, or narrowly scoped permission policy. Host-level administrative actions remain exceptional.

2.5 Multiple agents should have explicit responsibilities

Codex and Claude Code should not modify the same checkout simultaneously without coordination. Separate Git worktrees or task branches let one agent implement while another reviews or explores alternatives.

2.6 Conversation should produce durable knowledge

A useful voice session should be able to trigger a reusable Skill that creates a design note, whitepaper, README, or other artifact while the conversation continues. OpenAI documents Skills as reusable workflows supported in Codex; Skills can package instructions, resources, and scripts for repeatable work [3].

3. Proposed Architecture

The system uses four logical layers: the voice control plane, the remote connection layer, the Ubuntu execution plane, and the project artifact layer.

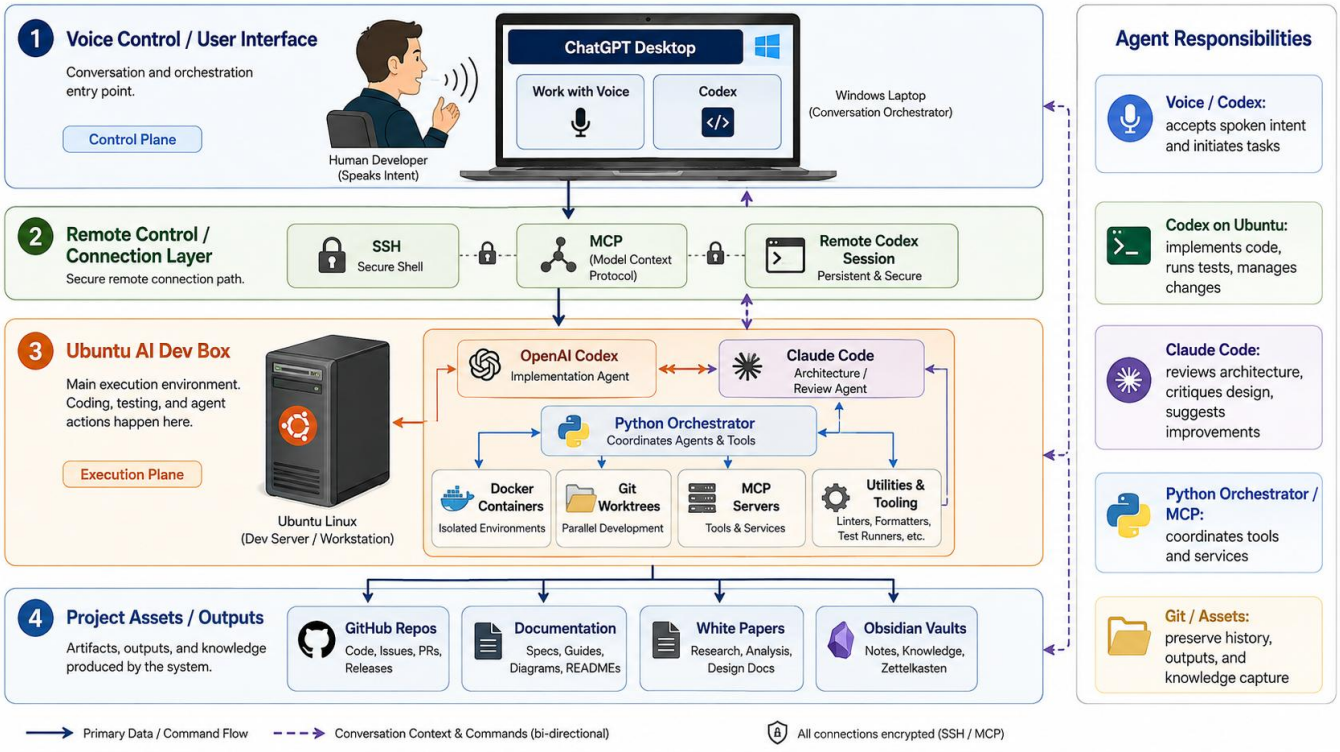

Figure 1: Proposed voice-controlled agentic development architecture. The Windows laptop provides voice control, Remote SSH connects to the Ubuntu execution plane, Codex and Claude Code operate in separated roles, and Git plus document stores preserve outputs.

The diagram shows a Python orchestrator and MCP servers as available coordination components. They are optional in the baseline deployment. The first implementation can use Codex itself as the primary orchestrator and add a local service only when repeated routing or automation requires one.

3.1 Control Plane: Windows Laptop

The Windows laptop runs the ChatGPT desktop application with Codex and Work available. The developer speaks to Voice and treats the conversation as the primary command surface. The control plane is responsible for four activities:

- receiving spoken intent;
- maintaining task context and asking clarifying questions;
- starting, steering, or reviewing remote agent work; and
- presenting diffs, test results, screenshots, and artifacts back to the developer.

The control plane should not become the primary build host. Keeping implementation work on the Ubuntu machine reduces configuration drift and preserves a stable development environment.

OpenAI's current documentation states that Voice in Work and Codex is a desktop capability on Windows and macOS. Voice uses the permissions and tools available to the selected experience [1]. This detail is important because voice does not create a new security model. A spoken command can only do what the underlying Codex or Work environment is already allowed to do.

3.2 Remote Connection Layer

Codex Remote SSH is the preferred path from the Windows control plane to the Ubuntu development box. OpenAI states that Remote SSH is generally available and that the desktop app can detect hosts from the user's SSH configuration [2]. This makes the remote Linux machine appear as a development environment rather than a second desktop that the user must manually operate through RDP.

SSH remains the machine-access transport. MCP serves a different purpose. An MCP server can expose structured tools, data, or local services to an agent, but it is not required to make the remote development box reachable. The initial architecture should therefore treat MCP as an optional tool-integration layer.

Remote desktop remains useful as a diagnostic path. It can help when the developer needs to inspect a local GUI, browser window, or machine state that the coding interface does not expose. It should not be the normal command path for agentic development.

[TODO: confirm that the school-managed network and host policy permit Remote SSH, agent installation, and the required outbound connections.]

3.3 Execution Plane: Ubuntu Development Box

The Ubuntu host is the primary execution environment. It contains the repositories, compilers, package managers, test runners, local databases, browsers, and other development tools required by the project.

The baseline host runs agents under an ordinary non-root user. The user may have sudo access, but sudo should require a human-controlled password. This prevents an autonomous session from silently converting project-level authority into machine-level authority.

The host also runs Codex and Claude Code. Codex acts as the primary implementation agent because the Windows control plane can directly manage Codex remote threads. Claude Code acts as a second agent that can review architecture, inspect code, run independent analyses, or implement bounded subtasks.

The integration between the two agents does not need a specialized cross-agent protocol. Claude Code is a command-line program; Codex can invoke command-line tools when the execution policy allows it. The architecture can therefore treat Claude Code as a subprocess or separately managed session. This is an integration pattern rather than a claim of a first-party Codex-to-Claude feature.

3.4 Project Artifacts and State

The bottom layer contains the durable outputs of the system. Git repositories store source and commit history. Markdown and Word documents store technical writing. An Obsidian vault or similar knowledge store can preserve design notes and research context.

This layer matters because conversational state is transient compared with project artifacts. The system should promote useful outcomes from conversation into files that can be reviewed, diffed, versioned, and reused by later agents.

4. Agent Roles and Coordination

4.1 Codex as Primary Orchestrator

The revised architecture places Codex in two related roles. On the Windows laptop, Codex provides the visible task interface through the desktop application and Voice. On the Ubuntu box, the remote Codex thread works inside the development environment.

This arrangement collapses an earlier idea that required a separate Python orchestrator for every task. A separate service is useful only when the workflow needs deterministic routing, scheduled jobs, persistent event queues, or coordination that should continue independently of a Codex conversation.

A typical spoken request might be:

> *Review the current branch, run the failing tests, ask Claude Code to critique the architecture, then propose the smallest change that fixes the performance regression. Do not modify production configuration.*

Codex can translate that goal into shell commands, test execution, repository inspection, and a Claude Code invocation. The developer remains in the voice conversation and can redirect the task when a decision point appears.

4.2 Claude Code as Reviewer and Secondary Implementer

Claude Code provides an independent agent with a different model family and tool harness. That diversity can be useful for design review because the second agent does not share the exact same reasoning path as the first.

The default role should be review-oriented: inspect Codex's changes, challenge assumptions, identify missing tests, and propose alternatives. Claude Code may also implement a bounded subtask, but parallel editing should use a separate worktree or branch.

A simple directory layout could be:

```
~/projects/my-app/                  # main working tree
~/worktrees/my-app-codex/           # Codex implementation worktree
~/worktrees/my-app-claude-review/   # Claude review or alternate implementation
```

Codex can merge selected changes after review. Git therefore serves as a coordination and recovery mechanism rather than a message bus.

4.3 Optional Python Orchestrator

A local orchestrator becomes useful when the system gains repeated multi-agent workflows. Examples include:

- run Codex implementation, then Claude review, then tests, then generate a report;
- capture every completed voice session into a project note;
- route research tasks to one agent and coding tasks to another; or
- enforce a state machine with required approval gates.

The orchestrator should remain small. It can expose commands through a local CLI, HTTP endpoint bound to localhost, or MCP server. Codex can then call it as a tool. The service should not duplicate the reasoning capabilities already present in the agents.

5. Privilege and Containment Model

The most important operational decision is where to place unrestricted execution. The design uses three levels of authority.

Level	Environment	Expected authority	Human approval
1	Windows control plane	Conversation, review, task steering	Normal desktop permissions
2	Ubuntu host user	Repository edits, builds, tests, user-space tools	Required for host sudo
3	Disposable container or sandbox	Broad project-local command execution	May use high-autonomy agent mode inside boundary

5.1 Avoid Passwordless Sudo on the Host

Passwordless sudo for the daily agent user would allow a coding model to convert any mistaken or malicious command into root-level host modification. The operational benefit is small because most development work does not require root after the machine is bootstrapped.

The developer should perform the initial host setup, including Docker or Podman installation, SSH configuration, and core build dependencies. After that, project-specific dependencies should live in user space, containers, virtual environments, or repository-controlled setup scripts.

For rare repeated administrative actions, a narrow sudoers rule can allow a specific command rather than all commands. The rule should name the exact executable and arguments where possible.

5.2 Claude Code Bypass Permissions

Anthropic documents a `bypassPermissions` mode, also exposed through `--dangerously-skip-permissions`, that skips normal permission prompts. Anthropic explicitly recommends using this mode only in isolated environments such as containers or virtual machines [4]. The documentation also notes that Claude Code refuses to start in this mode as root or under sudo on Linux and macOS unless it is inside a recognized sandbox [5].

This matches the proposed design. The Ubuntu host itself remains ordinary and protected. A disposable development container can run Claude Code with bypass permissions when the task benefits from uninterrupted autonomy.

The container should mount only the required repository or worktree. It should not receive the user's entire home directory, SSH agent socket, cloud credential directories, or unrelated project data. Network access should also be limited when the task does not require it.

5.3 Codex Sandbox and Approval Policy

OpenAI describes Codex as a system designed around sandboxing and approvals. The sandbox defines writable paths and network boundaries; the approval policy determines when the agent may cross those boundaries [6]. OpenAI's published internal deployment guidance favors constrained execution, managed network policy, rules for known-safe commands, and detailed agent logs [6].

The practical baseline is therefore workspace-scoped write access with normal approvals rather than permanent full access on the host. If a task repeatedly needs unsandboxed execution, that is evidence that the execution environment should be redesigned, usually by moving the task into a container with the required tools already installed.

5.4 Credentials and Network Access

Agents should receive the minimum credentials needed for the current task. Project tokens should be scoped and revocable. Long-lived personal credentials should not be copied into a container merely to remove prompts.

Network access deserves the same treatment. Package installation may require access to registries, and Git operations may require access to a code host. A container does not automatically make unrestricted network access safe. Prompt injection or mistaken automation can still reach external systems if the container has broad credentials and network access.

[TODO: identify whether any project code, data, or credentials are subject to Virginia Tech or project-specific restrictions on external AI processing.]

6. Performance and Resource Strategy

The development box is intended to provide more resources than the laptop, so isolation should not consume those resources without a reason. A full virtual machine provides a strong boundary, but it duplicates an operating system and reserves memory, storage, and device resources. That may be unnecessary for ordinary repository work.

Containers are a better default for this architecture. They share the host kernel and can provide process, filesystem, and network isolation with less overhead than a conventional full virtual machine. The exact performance impact depends on the workload, especially filesystem I/O, networking, GPU access, and bind mounts.

The existing application's slow behavior should therefore be profiled before changing the deployment model. The recommended sequence is:

- establish a native Ubuntu baseline;
- profile CPU, memory, I/O, database calls, browser rendering, and network latency;
- identify the dominant bottleneck;
- introduce containerization for agent isolation; and
- compare the containerized result against the baseline.

This avoids blaming virtualization for a performance problem that may already exist in the application.

A full VM remains useful for high-risk experiments that require kernel modules, system package modification, nested services, or a completely disposable operating system. It should be the exception rather than the default.

[TODO: capture a baseline profile of the slow web application before selecting the final isolation strategy.]

7. Voice as an Operational Interface

Voice becomes valuable when it controls a durable task state rather than a transient command. OpenAI's current desktop implementation supports speaking to Work or Codex, interrupting naturally, and asking Voice to start or coordinate tasks [1]. Codex Remote SSH allows those tasks to operate where the files and tools reside [2].

That combination supports a workflow in which the developer can remain conversational while the system performs longer operations. For example:

- **Step 1:** The developer says, "Open the visualization project on the school dev box and reproduce the slow page load."
- **Step 2:** Codex connects through Remote SSH and runs the application, tests, or profiler.
- **Step 3:** Codex reports the first findings in the same voice conversation.

- **Step 4:** The developer says, "Have Claude review that diagnosis before changing anything."
- **Step 5:** Codex invokes Claude Code in the review worktree and summarizes disagreements.
- **Step 6:** The developer chooses a direction.
- **Step 7:** Codex implements the change and runs validation.
- **Step 8:** The developer says, "Capture what we learned as a design note and update the project README."

The key interaction is intent plus review. Voice does not need to dictate every shell command. The agent can translate a goal into operations while the user retains authority over scope and consequential decisions.

8. Knowledge Capture as a First-Class Workflow

The original frustration in the conversation was the need to export a transcript, open a separate session, and manually invoke a writing workflow. The architecture can remove most of that ceremony by installing the same reusable Skills in Codex.

OpenAI documents that Skills are supported in Codex and can be explicitly invoked or selected automatically for relevant tasks [3]. Personal Skills do not necessarily sync automatically across products or surfaces, so the `chat-to-whitepaper` Skill should be installed in the Codex environment that will execute the workflow [7].

Once installed, the voice control plane can treat knowledge capture as another task. A spoken request such as "turn this discussion into a whitepaper using my chat-to-whitepaper skill" can start the writing workflow without manually exporting the transcript first, provided the active Codex session has access to the relevant conversation context or a captured summary.

The implementation should preserve a simple artifact pipeline:

```
voice discussion
    -> structured task/context capture
    -> chat-to-whitepaper skill
    -> Markdown source
    -> formatted Word whitepaper
    -> Git or knowledge repository
```

The same pattern can support other durable outputs. A session can create a design decision record, issue, experiment log, or research note. The value comes from making capture part of the normal conversational interface rather than a cleanup task performed later.

A future local orchestrator could automate this further. For example, a `capture-session` command could assemble a transcript summary, record the active Git commit, list modified files, and invoke the appropriate writing Skill. The resulting document would then include both the design reasoning and the state of the implementation that produced it.

9. Recommended Deployment Sequence

The architecture should be built incrementally so each layer can be tested before adding autonomy.

Phase 1: Prepare the Ubuntu Host

Install Ubuntu, create a normal development user, configure SSH keys, and install the minimum host-level tools. Install Git and the container runtime manually. Do not enable blanket passwordless sudo.

Create a top-level project layout that separates repositories, worktrees, and agent scratch space. Keep unrelated personal data outside the directories exposed to agents.

Phase 2: Establish Remote Codex

Install and authenticate Codex on the Windows laptop. Configure the Ubuntu host in the Windows SSH configuration. Verify that Codex Remote SSH can open the repository, read files, run a harmless command, and return output.

Keep the initial approval policy conservative. Confirm that ordinary repository edits and tests work before enabling broader permissions.

Phase 3: Add Voice

Enable Voice in Codex or Work on the Windows desktop application. Use short operational tasks first: inspect status, run a test, explain a failure, or create a branch. Verify that the spoken request, remote task, and returned results remain understandable in one conversation.

Phase 4: Install Claude Code

Install Claude Code on the Ubuntu host under the normal development user. Start with its standard permission mode. Create a separate review worktree and test a simple review task.

Only after the container boundary is working should `bypassPermissions` be tested, and then only inside the isolated development container.

Phase 5: Add Reusable Skills

Install the writing and knowledge-capture Skills needed for repeatable workflows. Test the `chat-to-whitepaper` Skill from Codex with a short synthetic conversation. Confirm that the generated Markdown and Word files land in the intended project or knowledge repository.

Phase 6: Add Automation Only Where Repetition Appears

If the same routing sequence is repeated often, add a small local orchestrator or MCP server. Examples include automatic review loops, session capture, test pipelines, or agent handoffs. Avoid introducing an orchestration service before a repeated need exists.

10. Threat Model and Mitigations

10.1 Privilege and Data Risks

Risk	Failure mode	Primary mitigation
Agent obtains host root	Mistaken or malicious command modifies system configuration or unrelated data	No passwordless sudo; ordinary host user; narrow admin rules
Broad autonomous file access	Agent edits unrelated repositories or personal files	Dedicated project directories; workspace write boundaries; container mounts limited to one worktree
Credential leakage	Prompt or tool output exposes tokens	Scoped credentials; avoid mounting home directory and SSH secrets; redact logs where needed
Prompt injection through external content	Agent follows hostile instructions from fetched content	Restrict network access; require approvals for external actions; separate read and write capabilities

10.2 Operational and Access Risks

Risk	Failure mode	Primary mitigation
Agents overwrite each other's work	Concurrent edits produce conflicts or lost changes	Separate Git worktrees and branches; explicit merge step
Destructive command	Agent removes project or system data	Git checkpoints; snapshots where needed; container boundary; host permissions
Excessive resource use	Long-running agent consumes CPU, memory, disk, or network	Container limits, process monitoring, quotas, task timeouts
Knowledge disappears in chat	Useful reasoning cannot be found or reused	Skills that produce versioned Markdown, whitepapers, ADRs, and README updates
Remote access exposes host	Internet-facing SSH or weak credentials create a new attack surface	Institutional VPN or approved SSH path, key authentication, firewall rules, Codex Remote SSH configuration

These risks reinforce a general rule: the system should make low-risk local work easy and make scope expansion visible. Autonomy is most useful when the environment is designed so that a mistaken action remains recoverable.

11. Operational Principles

Several concise rules capture the intended behavior of the system.

Keep the laptop as the control plane. It handles voice, review, and human decisions. The source of truth remains on the Ubuntu machine.

Keep the Ubuntu host ordinary. Agents run as a normal user. Host administration remains a human-controlled boundary.

Put high-autonomy modes in disposable environments. Claude Code bypass permissions and Codex full-access behavior should be reserved for sandboxes, containers, or environments that can be rebuilt.

Use Git checkpoints before long autonomous tasks. The agent should be able to show what changed and the developer should be able to revert it.

Use separate worktrees for competing agents. Parallel work is useful when the merge is explicit.

Treat MCP as an integration mechanism. Use it when an agent needs structured access to a service or tool; do not add it merely to create a remote shell path.

Capture reasoning into files. A completed voice session should be able to produce a design note, whitepaper, issue, or code change that remains useful after the conversation ends.

12. Open Questions

Three deployment questions remain outside the scope of the current conversation.

First, the school environment may impose restrictions on remote access, software installation, AI services, or storage of project data. Those policies can change the allowed connection and credential model.

Second, the appropriate container boundary depends on the actual application. A CPU-bound backend, an I/O-heavy visualization pipeline, and a GPU application have different isolation costs. Profiling should determine whether the default container design needs adjustment.

Third, the most useful knowledge-capture trigger still needs implementation detail. Codex can use Skills, but the exact method for selecting and packaging the relevant conversation context should be tested in the intended desktop and remote workflow. The simplest implementation may be an explicit spoken command that asks Codex to summarize the current thread and invoke the installed Skill.

13. Conclusion

A voice-driven development environment works best when voice controls a bounded execution system rather than a fully privileged desktop. The Windows laptop can serve as the conversational control plane; Codex Remote SSH can connect that interface to a persistent Ubuntu development box where the code, tools, and compute already exist. Codex can perform implementation work and coordinate other command-line agents. Claude Code can provide independent review or parallel implementation in a separate worktree.

The security model follows the same separation. The Ubuntu host retains normal user and sudo boundaries. High-autonomy agent modes belong inside containers or other disposable sandboxes. Git supplies checkpoints and reviewable history. Skills turn useful conversations into durable artifacts so the design reasoning does not disappear when the voice session ends.

This architecture reduces the friction between thinking, directing, implementing, and documenting. It also keeps those activities inspectable. The human remains responsible for goals, scope, and consequential approvals; the agents translate those decisions into repeatable technical work.

References

[1] OpenAI, "ChatGPT Work and Codex," OpenAI Help Center, updated August 2026. https://help.openai.com/en/articles/20001275/

[2] OpenAI, "Work with Codex from anywhere," May 14, 2026. https://openai.com/index/work-with-codex-from-anywhere/

[3] OpenAI, "Introducing the Codex app," 2026. https://openai.com/index/introducing-the-codex-app/

[4] Anthropic, "Configure permissions," Claude Code Docs, 2026. https://code.claude.com/docs/en/permissions

[5] Anthropic, "Choose a permission mode," Claude Code Docs, 2026. https://code.claude.com/docs/en/permission-modes

[6] OpenAI, "Running Codex safely at OpenAI," May 8, 2026. https://openai.com/index/running-codex-safely/

[7] OpenAI, "Skills in ChatGPT," OpenAI Help Center, updated August 2026. https://help.openai.com/en/articles/20001066